

Reviewer Pack — Minimal Representation Rank of Permutation Groups for Commun...

Entry 304262ad4953 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-21 16:30:33 UTC

Minimal Representation Rank of Permutation Groups for Communication Complexity Lower Bounds

Entry ID: 304262ad4953 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-21 16:30:33 UTC

1. Conjecture

Field A (mathematical branch): Group Theory: Permutation Groups **Field B** (complexity object): Communication Complexity

Statement:

[‘For all instances of the DISJOINTNESS problem with n variables, if a permutation group G has minimal representation rank $r(G)$ such that G can be used to encode a communication protocol for DISJOINTNESS with complexity $O(n^{\{1/r(G)\}})$, then $n \leq 40$.’, ‘For all instances with the above property, there exists a permutation group G with $r(G) \leq 5$ and a communication protocol with complexity $O(n^{\{1/r(G)\}})$ that can be used to solve DISJOINTNESS.’, ‘If there is an instance of DISJOINTNESS for which no permutation group G with $r(G) \leq 5$ can encode a communication protocol with complexity better than $O(n^{\{0.8\}})$, then $n > 40$.’]

Rationale (proposer’s reasoning):

[‘Permutation groups are well-studied in group theory and their representation ranks provide a measure of the complexity of the underlying permutations. Communication complexity is a well-established area in complexity theory that measures the amount of communication needed to solve certain problems.’, ‘By relating the representation rank of permutation groups to communication complexity, we may uncover new structural insights into both fields.’, ‘This conjecture suggests a potential connection between abstract algebra and concrete computational tasks, which has not been extensively explored.’]

Taxonomy category: GROUP_THEORY_PERMUTATION_GROUP_COMMUNICATION_COMPLEXITY (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): b276cef6e0ca4a88

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if all instances with n variables ($n \leq 40$) have a permutation group G with $r(G) \leq 5$ that encodes a communication protocol for DISJOINTNESS with complexity $O(n^{\{1/r(G)\}})$ and no instance has a permutation group G with $r(G) \leq 5$ encoding a protocol with complexity greater than $O(n^{\{0.8\}})$. The conjecture is falsified if any instance with n variables ($n \leq 40$) has no permutation group G with $r(G) \leq 5$ encoding a protocol with complexity better than $O(n^{\{0.8\}})$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.90	SAFE	UNCERTAIN
KARP_LIPTON	SAFE	1.00	SAFE	SAFE

4. Novelty audit

Verdict: ADJACENT_OK against 1 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "minimal representation rank" AND "permutation groups" AND "communication complexity" - "DISJOINTNESS problem" AND "permutation group G " AND "complexity $O(n^{\{1/r(G)\}})$ " - "encoding protocol" AND " $r(G) \leq 5$ " AND "solving DISJOINTNESS"

Top relevant hits considered: - [s2:537b8f418572696a834ee62011e1dd55477b3ba3] Cancer Therapy : Preclinical

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=0, elapsed=0.0s

5.1 Generated Python source

```
from fractions import Fraction
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def communication_protocol_complexity(n, r):
        if r == 0:
            return float('inf')
        return n**(1/r)

    def minimal_representation_rank(G):
        # Placeholder for actual computation of minimal representation rank
        # For simplicity, we assume a fixed value for demonstration purposes
        return random.randint(1, 5)

    def generate_disjointness_instance(n):
        instance = [random.sample(range(n), n) for _ in range(n)]
        return instance
```

```

def find_permutation_groups(instance):
    # Placeholder for actual computation of permutation groups
    # For simplicity, we assume a fixed set of groups for demonstration purposes
    G = {frozenset(range(1, n+1)), frozenset(range(n, 0, -1))}
    return G

def check_protocol_complexity(G, instance):
    for g in G:
        r_G = minimal_representation_rank(g)
        protocol_complexity = communication_protocol_complexity(len(instance), r_G)
        if protocol_complexity <= len(instance)**(1/r_G):
            return True
    return False

n = random.randint(5, 40)
instance = generate_disjointness_instance(n)
G = find_permutation_groups(instance)

result = check_protocol_complexity(G, instance)

return {
    "metric_name": "Protocol Complexity",
    "metric_value": communication_protocol_complexity(n, minimal_representation_rank(frozenset(instance))),
    "instances_tested": 1,
    "conjecture_holds": result,
    "counterexample": "" if result else f"No permutation group with r(G) <= 5 found for n={n}"
}

if __name__ == "__main__":
    import sys
    seeds = [int(sys.argv[1])] if len(sys.argv) > 1 else list(range(2, 30)) # Default to first 29 seeds

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    if all(result["conjecture_holds"] for result in results):
        mean_value = sum(result["metric_value"] for result in results) / len(results)
        support_fraction = 1.0
        print(f"RESULT: SUPPORTED mean={mean_value} std=0 support_fraction={support_fraction}")
    elif any(not result["conjecture_holds"] for result in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"No permutation group with r(G) <= 5 found\" for n={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE mapping_undefined")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TRIAL: {'metric_name': 'Protocol Complexity', 'metric_value': 2.0123466170855586, 'instances_tested': 1}

RESULT: SUPPORTED mean=2.0123466170855586 std=0 support_fraction=1.0

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

The test only includes a single instance ($n = 1$), which is insufficient to draw conclusions about the conjecture’s validity for all instances of DISJOINTNESS with n variables. This is a common failure mode where ‘ n too small’ leads to trivial results that do not scale.

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test only includes a single instance ($n = 1$), which is insufficient to draw conclusions about the conjecture’s validity for all instances of DISJOINTNESS with n variables. The critic challenges the validity of the results based on this reason. | next: Conduct additional tests with varying values of $n \leq 40$ to validate the conjecture across a broader range of instances.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14823
2	preregistration	ollama_remote	glm4:latest	0	0	10944
3	novelty	ollama_remote	glm4:latest	0	0	8678
4	novelty	ollama_remote	glm4:latest	0	0	9768
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12550
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11480
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7656
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7712
9	critic	ollama_remote	glm4:latest	0	0	12390
10	judge	ollama_remote	glm4:latest	0	0	9137

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 105137 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/304262ad4953.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/304262ad4953.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/304262ad4953.tar.gz` (if generated)